Modul 2.4: Dateien lesen und bearbeiten

Anschluss an Modul 2.3

In Modul 2.3 hast du Verzeichnisse und Dateien angelegt, kopiert und verschoben. Jetzt liest du Dateien gezielt und änderst eine Konfiguration kontrolliert. Du arbeitest ausschließlich in einem eigenen Übungsbereich.

Lernziele und Praxisbezug

Nach diesem Modul kannst du:

- kurze Dateien mit `cat` vollständig lesen,
- lange Dateien mit `less` seitenweise lesen,
- mit `head` den Anfang und mit `tail` das Ende prüfen,
- versteckte Dateien mit `ls -a` finden,
- eine Textdatei mit `nano` ändern,
- vor und nach jeder Änderung den Inhalt prüfen.

Konfigurationsdateien bestimmen das Verhalten von Programmen. Logdateien zeigen, was passiert ist, und Dokumentation erklärt den Aufbau eines Projekts. Darum gilt: **Erst lesen, dann ändern, anschließend prüfen.**

> **Warning**
>
> Alle schreibenden Befehle und Änderungen in diesem Modul bleiben auf `~/ubuntu-schulung/dateien-labor` beschränkt. Die Datei `.env` enthält hier nur erfundene Beispielwerte. Ändere keine echten Konfigurationen und verwende kein `sudo`.

1. Übungsbereich und Beispieldateien erstellen

Lege den Arbeitsbereich selbst an und wechsle hinein:

```
mkdir -p ~/ubuntu-schulung/dateien-labor/{config,docs,logs,src}
cd ~/ubuntu-schulung/dateien-labor
```

Erstelle eine kurze Konfiguration, eine Dokumentation, eine versteckte Umgebungsdatei und eine längere Logdatei:

```
printf '%s\n' \
  'app_name=Webshop' \
  'port=8080' \
  'debug=false' \
  'log_level=info' \
  'max_connections=20' \
  > config/app.conf
printf '%s\n' \
  'Projekt: Webshop' \
  'Status: Testumgebung' \
  'Zustaendig: Entwicklung' \
  > docs/README.txt
printf '%s\n' \
  'APP_ENV=test' \
  'APP_SECRET=beispielwert-nicht-echt' \
  'DATABASE_URL=sqlite:///test.db' \
  > .env
printf '%s\n' \
  '08:01 GET / 200' \
  '08:02 GET /produkte 200' \
  '08:03 GET /login 200' \
  '08:04 GET /admin 401' \
  '08:05 GET /admin 401' \
  '08:06 GET /login 200' \
  '08:07 GET /produkte 200' \
  '08:08 GET /admin 401' \
  '08:09 GET / 200' \
  '08:10 GET /status 200' \
  '08:11 GET /produkte 200' \
  '08:12 GET /login 200' \
  > logs/access.log
```

Prüfe deinen Ort und die sichtbaren Einträge:

```
pwd
ls
```

`config`, `docs`, `logs` und `src` sollten sichtbar sein. `.env` fehlt noch, weil Namen mit einem Punkt standardmäßig verborgen werden.

2. Kurze Dateien mit cat lesen

`cat` zeigt den vollständigen Inhalt einer kurzen Datei:

```
cat config/app.conf
cat docs/README.txt
```

Du siehst Einstellungen im Format `schluessel=wert`. Bei kurzen Konfigurationsdateien ist `cat` schnell und übersichtlich. Bei langen Dateien würde die Ausgabe dagegen am Bildschirm vorbeilaufen.

3. Lange Dateien mit less lesen

Öffne das Protokoll seitenweise:

```
less logs/access.log
```

Probiere diese Tasten aus:

Taste	Wirkung
Leertaste	eine Seite vorwärts
`b`	eine Seite zurück
`q`	`less` beenden

Drücke am Ende `q`, um zur Shell zurückzukehren. Wenn das Terminal scheinbar keine Befehle annimmt, bist du wahrscheinlich noch in `less`.

4. Anfang und Ende mit head und tail

`head` zeigt standardmäßig die ersten zehn Zeilen, `tail` die letzten zehn:

```
head logs/access.log
tail logs/access.log
```

Bei Protokollen stehen neue Einträge häufig am Ende. Mit `tail` findest du deshalb schnell aktuelle Ereignisse. Begrenze die Anzahl mit `-n`:

```
head -n 3 logs/access.log
tail -n 3 logs/access.log
```

Die passende Auswahl hängt von der Frage ab:

Situation	Befehl
kurze Datei komplett	`cat datei`
lange Datei kontrolliert	`less datei`
erste N Zeilen	`head -n N datei`
letzte N Zeilen	`tail -n N datei`

5. Versteckte Dateien finden

Zeige alle Einträge einschließlich versteckter Namen:

```
ls -a
cat .env
```

Dateien, deren Name mit `.` beginnt, heißen **Dotfiles**. Das Verbergen dient der Übersichtlichkeit, nicht der Sicherheit. Wer Zugriffsrechte hat, kann `.env` lesen. In echten Projekten können dort Passwörter oder API-Schlüssel stehen; sie gehören nicht in öffentliche Dokumentation oder ungeschützte Versionskontrolle.

6. Eine Konfiguration mit nano bearbeiten

Ändere nur Dateien in deinem Übungsbereich. Lies die Konfiguration zuerst:

```
cat config/app.conf
```

Öffne sie anschließend:

```
nano config/app.conf
```

Navigiere mit den Pfeiltasten und ändere `debug=false` in `debug=true`. Speichere und beende den Editor:

1. `Strg+O` drücken, um zu speichern.
2. Den Dateinamen mit `Enter` bestätigen.
3. `Strg+X` drücken, um `nano` zu verlassen.

Prüfe danach den Inhalt:

```
cat config/app.conf
```

Die übrigen Zeilen sollen unverändert geblieben sein. Falls du die Änderung verwerfen möchtest, drücke in `nano` `Strg+X` und beantworte die Frage nach dem Speichern mit `N`.

7. Selbstständige Aufgabe: Konfiguration anpassen

Passe `config/app.conf` mit dem vollständigen Arbeitsablauf an:

1. Lies die Datei mit `cat`.
2. Ändere mit `nano` `log_level=info` in `log_level=debug`.
3. Ändere `max_connections=20` in `max_connections=25`.
4. Prüfe die Datei erneut mit `cat`.

Akzeptanzkriterien:

- `debug=true` ist weiterhin vorhanden.
- `log_level=debug` ist vorhanden.
- `max_connections=25` ist vorhanden.
- `app_name`, `port` und alle übrigen Werte sind unverändert.
- Die Datei liegt weiterhin unter `~/ubuntu-schulung/dateien-labor/config/app.conf`.

Transfer in den Arbeitsalltag

- **Systemintegration:** Lies Konfiguration und aktuelle Logzeilen, bevor du einen Dienst änderst oder neu startest.
- **Anwendungsentwicklung:** Prüfe vor einer Änderung, ob eine Einstellung lokal, für Tests oder für die Produktion gilt.
- **Daten- und Prozessanalyse:** Verwende `head`, `tail` oder `less`, um große Eingabedateien zunächst stichprobenartig und kontrolliert zu prüfen.

Zusammenfassung

Du kannst jetzt das passende Lesewerkzeug auswählen, Dotfiles entdecken und mit `nano` eine Konfiguration ändern. Der sichere Ablauf lautet: **lesen – gezielt ändern – erneut lesen und prüfen**.